Lab 16 — Questions

Question 1 [Understanding]

Explain the difference between `requests` and `limits` for memory and for CPU: who reads each one (scheduler, kubelet, cgroup), and what happens when a container exceeds its memory **limit** on the one hand, its CPU limit on the other?

Question 2 [Prediction]

Three Pods on a node starved of memory: `db` with `requests = limits = 512Mi`, `api` with `requests 64Mi / limits 256Mi`, `batch` with no `resources`. Give each one's QoS class, the order in which the kubelet evicts them, and explain why that is not the order the team would want for `batch` if it were a critical accounting job.

Question 3 [Diagnosis]

```
$ kubectl get pod glouton
NAME      READY   STATUS      RESTARTS      AGE
glouton   0/1     OOMKilled   1 (14s ago)   16s
```

Give the two commands that confirm the cause and the exceeded value, explain why `RESTARTS` grows, and say what has to be fixed in this case: the limit, the application, or both.

Question 4 [Diagnosis]

After `kubectl scale deployment api --replicas=6`, three Pods stay `Pending`. `kubectl describe node | grep -A6 'Allocated resources'` shows `memory 3800Mi (95%)` in requests. Explain, propose three different fixes, and say which one is the most honest.

Question 5 [Analysis]

A LimitRange with `defaultRequest: {memory: 32Mi}`, `default: {memory: 128Mi}` and `max: {memory: 1Gi}` is set in the namespace. Describe what happens to: (a) a Pod without `resources`; (b) a Pod with `limits.memory: 4Gi`; (c) a Pod with `requests.memory: 64Mi` and no limit. Why does the platform team impose these objects?

Question 6 [Prediction]

A ResourceQuota sets `requests.memory: 1Gi` and `pods: 10` on the namespace. A three-replica Deployment with `requests.memory: 64Mi` is running. You apply a second Deployment with eight replicas of `128Mi` each. What does `kubectl get pods` show for that second Deployment, what does `kubectl describe deployment` say, and where do you read the refusal?

Question 7 [Understanding]

Why does `kubectl logs api-xxx` sometimes show healthy logs for a Pod that has just restarted, while the cause of the restart is in `kubectl logs api-xxx --previous`? What does `kubectl get pod -o jsonpath='{.status.containerStatuses[0].lastState}'` show?

Question 8 [Analysis]

The API image is distroless (lab 05): no shell, no `wget`. A Pod answers slowly and you want to see its processes and test its port from the inside. Give the full `kubectl debug` command, explain

what `--target` brings, and say why this approach is preferable to "adding a shell to the image just in case".

Question 9 [Diagnosis]

```
$ kubectl get secrets --as=system:serviceaccount:lab16:reader
Error from server (Forbidden): secrets is forbidden: User "system:serviceaccount:lab16:reader"
cannot list resource "secrets" in API group "" in the namespace "lab16"
```

The ServiceAccount `reader` has a Role on `pods` and `pods/log`. Explain each part of the message, say whether it is a bug or the intended behaviour, and give the rule to add if `reader` had to read ConfigMaps (not Secrets).

Question 10 [Analysis]

Compare Role + RoleBinding with ClusterRole + ClusterRoleBinding, then the mixed case ClusterRole + RoleBinding. For each, give a concrete use in a company, and explain why the `view` / `edit` / `admin` ClusterRoles are almost always used through RoleBindings.

Question 11 [Prediction]

```
kubectl apply -f netpol-api.yaml       # podSelector app=api, ingress from podSelector app=web,
port 8080
kubectl exec client -- wget -qO- --timeout=3 http://api:8080/actuator/health
kubectl label pod client app=web
kubectl exec client -- wget -qO- --timeout=3 http://api:8080/actuator/health
```

Predict the two results, explain what "selected by a policy" changes, and say what would happen on a cluster whose CNI does not enforce NetworkPolicies.

Question 12 [Analysis]

The security team asks that `db` be reachable only from `api`, that `api` be reachable only from `web` and from Traefik, and that nothing else changes. Write the principle of the required policies (no full YAML), say where the difficulty lies for Traefik (hint: namespace), and why one often starts with a "deny everything" policy.

Question 13 [Analysis]

An API Pod was started with the `default` ServiceAccount and, after a vulnerability, an attacker gets code execution inside it. What do they gain from the token mounted in the Pod on a cluster where `default` has no RoleBinding? And on a cluster where a hurried administrator had bound `default` to `cluster-admin` "so that the deployment works"? Which two measures from the lab would have reduced the risk?